

Zeyla — 24-Hour Hackathon Architecture

1. Guiding principles

- **One deployable, not microservices.** A modular monolith (Node.js/Express or FastAPI) on Render, with clear internal module boundaries (`auth/`, `marketplace/`, `escrow/`, `realtime/`, `notifications/`). You can split it later; you cannot afford service-to-service auth and deploy pipelines tonight.
- **Managed auth, not homemade.** Use Supabase Auth or Firebase Auth for login/session/JWT. Roll your own KYC *on top* of it — don't build password resets from scratch.
- **Postgres does more than you think.** With the PostGIS extension it handles geospatial queries (nearest provider, radius search) without a separate geo database.
- **Fake what the judges won't test.** Real-time GPS from a phone, live Telebirr settlement, and full biometric liveness detection are all things you can convincingly simulate. Spend real engineering time on the escrow state machine and trust score, since that's the core IP of "Zeyla."

2. Core tech stack

Layer	Choice	Why
Backend	Node.js + Express (or FastAPI)	Fast to scaffold, huge ecosystem, one repo
DB	PostgreSQL + PostGIS (Render managed)	Relational + geo in one place
Cache / pub-sub	Redis (Render managed)	Live location cache, ping fan-out
Realtime	Socket.io (or native WS) over Redis adapter	Provider↔user live map + ping notifications
Auth	Supabase Auth / Firebase Auth	Login, sessions, JWT — don't hand-roll
Frontend	React (Vite) as a PWA	One codebase for web + installable mobile-like experience
Hosting	Render (web service + Postgres + Redis)	You already have credits
Payments	Telebirr API	Required by spec

3. Where your credits actually plug in

Tool	Feature it powers	Why it fits	Priority
Telebirr API	Escrow funding & payout	Core requirement — customer pays into escrow before contract starts, released to provider on completion	Must-have
Fal	ID-card OCR + selfie-vs-ID face match at signup	Fast hosted inference, no GPU setup — turns "identity verification" from a checkbox into a real demo moment	Must-have (it's your headline differentiator)
Whisperflow	Voice-based service requests ("find me an electrician near Bole") and voice reviews	Speech-to-text lets low-literacy or busy users use the platform hands-free — strong demo beat	High
ElevenLabs	Voice ping alerts to providers, and reading reviews/trust-score summaries aloud	Turns a silent push notification into something judges <i>hear</i> on stage	High
Addis AI	Amharic/English NLP: parsing voice-transcribed requests into structured search filters, moderating reviews and flags, summarizing why a trust score is what it is	Local-language support is a genuine differentiator for an Ethiopian marketplace	Medium
Firecrawl	Enriching provider profiles by pulling public business info (e.g. a Facebook page or registry listing) to corroborate identity/experience claims	Boosts trust score credibility with real external signal, cheap to wire up	Nice-to-have (cut first if short on time)
Render	Hosting backend, Postgres, Redis	Infra	Must-have

Sequencing rule of thumb: build Telebirr escrow and Fal ID-verification first — they're both "must-have" and both touch money/identity, the two things judges scrutinize hardest.

Voice (Whisperflow + ElevenLabs) is your polish layer — add once the core loop works end to end.

4. Data model (core tables)

```
users          (id, phone, name, role[user|provider], id_doc_url,
selfie_url,
               kyc_status, created_at)
providers      (user_id FK, category, bio, experience_years, base_lat,
base_lng,
               trust_score, is_online, current_lat, current_lng,
updated_at)
service_requests (id, user_id FK, category, lat, lng, status[pending|pinged|
               accepted|in_progress|completed|cancelled], created_at)
pings          (id, request_id FK, provider_id FK,
status[sent|seen|accepted|
               declined], sent_at)
contracts      (id, request_id FK, provider_id FK, user_id FK,
agreed_amount,
               status[awaiting_escrow|escrowed|active|completed|disputed],
               created_at, completed_at)
escrow_ledger  (id, contract_id FK, amount, telebirr_ref,
status[pending|held|
               released|refunded], created_at, released_at)
reviews        (id, contract_id FK, rating, comment, voice_url, created_at)
flags          (id, provider_id FK, flagged_user_id FK, reason, created_at)
trust_score_log (id, provider_id FK, delta, reason, created_at)
```

5. Key flows

Onboarding & KYC

1. Phone auth via Supabase/Firebase → OTP.
2. User uploads ID photo + selfie → sent to Fal for OCR + face-match score.
3. `kyc_status` set to `verified` if match score clears threshold, else `manual_review` (demo: just auto-pass above a hardcoded confidence).

Discovery & ping

1. User either browses provider list (filtered by category, distance via PostGIS `ST_DWithin`, trust score) or posts a request.
2. Optional: user speaks the request → Whisperflow transcribes → Addis AI extracts `{category, urgency, location}` → pre-fills the search form.

3. User pings a provider → row in `pings`, event pushed over Socket.io to that provider's room; if provider is offline, queue it and also fire an ElevenLabs-generated voice notification (or just a rich push).

Realtime tracking

1. Provider app posts `{lat, lng}` every ~5-10s over WebSocket once a contract is `active`.
2. Backend writes to Redis (`geo:contract:{id}`, short TTL) and broadcasts to the paired user's socket room.
3. Frontend renders both pins on a Leaflet/Mapbox map, polling the socket stream — no need to persist every ping to Postgres, only log start/end points.

Escrow + Telebirr payment

1. Contract created in `awaiting_escrow`.
2. Customer triggers Telebirr payment request (STK-style push) for `agreed_amount`.
3. On Telebirr webhook/callback confirming payment, write `escrow_ledger` row with `status=held`, flip contract to `escrowed`, then `active` once provider confirms start.
4. On job completion confirmed by both parties, backend calls Telebirr payout to provider (minus a platform fee), `escrow_ledger.status=released`.
5. If flagged/disputed before completion, funds stay `held`; add a manual admin-release endpoint for demo purposes rather than building a full dispute UI.

Reviews & flags

- Review only unlockable after `contract.status=completed`.
- Voice reviews: Whisperflow transcribes, Addis AI runs light sentiment/toxicity check before it's stored.
- Provider can flag a user post-contract with a reason; repeated flags reduce that user's ability to ping new providers (simple threshold, not a full moderation queue).

Trust score (simple, explainable formula for the demo)

```
trust_score = base(50)
+ completed_contracts * 2          (cap +20)
+ avg_review_rating_normalized      (0-20)
- flags_received * 5                (floor 0)
+ verified_kyc ? 10 : 0
+ firecrawl_profile_match ? 5 : 0
```

Recompute on write (trigger or simple background job), store history in `trust_score_log` so you can show a "why this score" breakdown — Addis AI can turn that breakdown into a

one-line natural-language explanation, which demos very well.

6. Suggested 24-hour build order

1. **Hrs 0-2:** Repo scaffold, Render deploy pipeline, Postgres + PostGIS + Redis provisioned, Supabase/Firebase auth wired.
2. **Hrs 2-6:** User/provider models, profile CRUD, provider discovery list + map (static location first).
3. **Hrs 6-9:** Ping flow + Socket.io realtime, contract state machine.
4. **Hrs 9-13:** Telebirr escrow integration (this is your riskiest external dependency — start it early, sandbox first).
5. **Hrs 13-16:** Fal ID/selfie verification wired into onboarding.
6. **Hrs 16-19:** Reviews + flags + trust score calculation and display.
7. **Hrs 19-22:** Whisperflow voice search, ElevenLabs voice ping alerts, Addis AI request-parsing and score explanations.
8. **Hrs 22-24:** Firecrawl profile enrichment if time allows; otherwise polish, seed demo data, rehearse the demo script.

7. What to fake if you're short on time

- **Live GPS:** simulate provider movement with a script animating along a fixed route instead of relying on real device GPS in a demo room.
- **Telebirr settlement:** use sandbox/test credentials and hardcode the webhook response if the live callback is flaky — the escrow *state machine* is what matters, not real money movement.
- **Firecrawl enrichment:** first tool to cut. A hardcoded "verified via public profile" badge tells the same story.
- **Dispute resolution:** one manual admin button, not a workflow.
- **KYC manual review queue:** skip it — auto-approve above a confidence threshold, auto-reject below it.